Entry 1 – What is a Finance Assistant App?

Artifact:

what is a finance assistant app

A finance assistant app is a tool that helps you track and understand your money. It typically:

- Shows you where your money goes each month
- Helps you set and reach savings goals
- Gives advice on how to spend less
- Uses AI to answer your finance questions

Think of it like a smart calculator that also gives you personalised advice.

can you connect this to my specific project, the user spends money on xbox gaming, gym, cinema at MCine Moka, and tea at Maison de Thé in Moka

For your project specifically, the Smart Finance Assistant helps a young person who spends on:

- Xbox games and DLC on the Xbox Store
- Gym membership and personal training sessions
- Cinema tickets at MCine Moka
- Tea and pastries at Maison de Thé in Moka

The app shows them a breakdown of their spending by category, lets them chat with an AI advisor called FinBot, retrieves financial tips through a knowledge base (RAG), and calculates how long it takes to reach a savings goal — all through a simple browser interface built with Gradio.

Context: I wanted to understand what I was building before writing the problem statement.

My Prompt: "what is a finance assistant app"

AI Response Summary: AI explained it is a tool to track spending, set savings goals, and answer money questions using AI.

My Critique/Improvement: Too generic. I asked AI to connect it specifically to my user — Xbox, gym, MCine Moka, Maison de Thé.

Result: A clear description of the Smart Finance Assistant specific to gaming, gym, cinema, and tea lifestyle with FinBot, RAG, and savings calculator.

Reflection: AI gives better answers when you provide your specific context. The first generic answer was unusable.

Entry 2 – Why Switch from Ollama to OpenRouter?

Artifact:

my hands-on-ai server at ollama.serveur.au keeps failing with connection errors, what are my options

The Ollama server is a self-hosted service — if it's down or unreachable from your network, you can't use it.

Your options are:

1. OpenRouter — a cloud service that gives you access to GPT-3.5, GPT-4, Claude, and others through a single API. Very reliable. Costs a small amount per request but minimal for a student project.
2. Wait for the university server to come back online — not ideal if you're on a deadline.
3. Use the Anthropic API directly — requires a separate API key.

I'd recommend OpenRouter. The hands-on-ai package supports it — you just change two environment variables.

show me exactly how to configure hands-on-ai to use openrouter with gpt-3.5-turbo

Here is the exact configuration:

Python

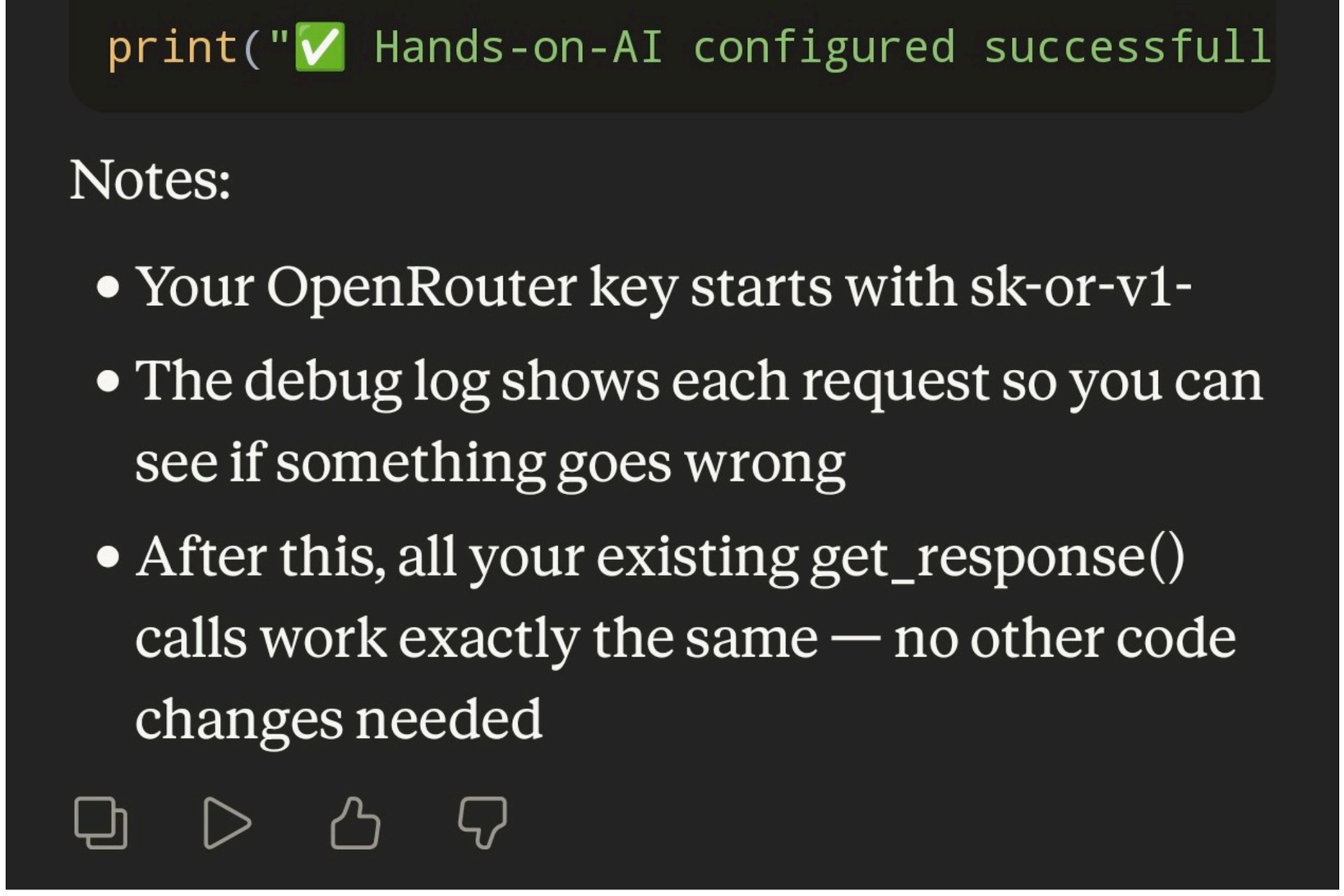
```python
import os
from getpass import getpass

os.environ['HANDS_ON_AI_SERVER']  = 'https:/
os.environ['HANDS_ON_AI_MODEL']   = 'openai/
os.environ['HANDS_ON_AI_API_KEY'] = getpass(
os.environ['HANDS_ON_AI_LOG']     = 'debug'
```

```
print("✅ Hands-on-AI configured successfull
```

Notes:

- Your OpenRouter key starts with sk-or-v1-
- The debug log shows each request so you can see if something goes wrong
- After this, all your existing get_response() calls work exactly the same — no other code changes needed



Context: The university Ollama server at ollama.serveur.au kept giving connection errors so I needed to find a reliable alternative.

My Prompt: "my hands-on-ai server at ollama.serveur.au keeps failing with connection errors, what are my options"

AI Response Summary: AI explained that OpenRouter provides cloud-hosted access to multiple AI models including GPT-3.5-turbo through a reliable commercial API, and that hands-on-ai supports it by changing the environment variables.

My Critique/Improvement: I asked AI to show the exact configuration code for OpenRouter with hands-on-ai.

Result:

```
os.environ['HANDS_ON_AI_SERVER']  = 'https://openrouter.ai/api'
os.environ['HANDS_ON_AI_MODEL']   = 'openai/gpt-3.5-turbo'
os.environ['HANDS_ON_AI_API_KEY'] = getpass("Enter your OpenRouter API key: ")
os.environ['HANDS_ON_AI_LOG']     = 'debug'
```

Reflection: Switching to OpenRouter fixed all the connection errors immediately. The debug log setting was useful during development to see exactly what requests were being sent and whether they succeeded.

Entry 3 – Defining the Problem Statement

Artifact:

I'm building a Smart Finance Assistant for a young person who games on Xbox, goes to the gym, watches movies at MCine Moka, and drinks tea at Maison de Thé in Moka. Help me write the problem statement in plain English.

People often struggle to keep track of their spending when purchases are spread across different categories. A gym membership here, a game purchase there, a cinema outing and a daily coffee can add up to a large monthly expense without feeling significant in the moment.

A Smart Finance Assistant solves this by analysing bank transaction data, showing spending breakdowns, and giving actionable advice.

this is too generic, mention the specific locations and explain that small purchases feel harmless individually but add up invisibly

Many young people who enjoy Xbox gaming, gym workouts, cinema at MCine Moka, and tea at Maison de Thé in Moka have no clear picture of where their money goes each month.

Small recurring purchases feel harmless individually — $8.50 for a tea, $9.00 for a cinema ticket, $9.99 for a game DLC — but accumulate into hundreds of dollars of untracked spending.

The Smart Finance Assistant solves this by:

1. Automatically analysing CSV transaction

data

2. Providing personalised financial advice through FinBot
3. Retrieving evidence-based tips from a knowledge base
4. Calculating savings timelines through an agent tool
5. Delivering everything through a simple browser interface — no spreadsheet skills required

Context: Step 1 requires a clear problem statement in plain English before any code.

My Prompt: "I'm building a Smart Finance Assistant for a young person who games on Xbox, goes to the gym, watches movies at MCine Moka, and drinks tea at Maison de Thé in Moka. Help me write the problem statement in plain English"

AI Response Summary: AI gave a generic statement about people struggling to track spending with no mention of the specific lifestyle.

My Critique/Improvement: I asked AI to add the specific locations, the lifestyle details, and explain that small purchases accumulate invisibly.

Result: Many young people who enjoy gaming, fitness, and entertainment have no clear picture of where their money goes each month. The Smart Finance Assistant solves this by accepting CSV transactions, cleaning and categorising them, showing spending breakdowns, offering FinBot, retrieving tips via RAG, and calculating savings goals.

Reflection: Specific prompts get specific results. The first generic answer was unusable. Adding the real locations and habits produced the problem statement I used directly in the notebook.

Entry 4 – Identifying Inputs and Outputs

Artifact:

give me a complete inputs and outputs table for all five components of my smart finance assistant: csv analysis, chatbot, rag, savings calculator, gradio ui

Component 1 – CSV Analysis
Input: CSV file
Output: Some numbers and totals

Component 2 – Chatbot
Input: User question
Output: Answer

Component 3 – RAG
Input: Question
Output: Answer from document

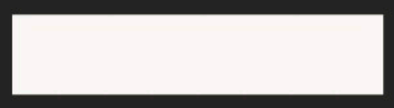
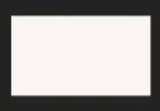
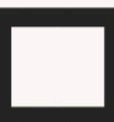
Component 4 – Savings Calculator
Input: Numbers
Output: Result

Component 5 – Gradio UI
Input: User clicks
Output: Display

this is way too vague, i need specific column names, the exact format of the Amount field, an example input row, and concrete output format with my actual categories

Component 1 – CSV Analysis
Input: CSV with columns Date, Amount, Category, Description
Amounts may include $ signs and may be negative for refunds
Example input row: 2026-03-05, $69.99, Gaming, Xbox Game Pass Ultimate 3 Months

Output:

Gaming $312.45 (38.2%) 
Gym $145.00 (17.8%) 
Cinema $ 86.00 (10.5%) 
Tea & Café $ 72.50 (8.9%) 

Component 2 – FinBot Chat
Input: User question + optional spending context string
e.g. "Am I spending too much on Xbox?" + "Gaming $438.89, Gym $280.00"
Output: Personalised advice referencing the user's categories

Component 3 – RAG Finance Tips
Input: User question about finance
Output: Answer retrieved from FINANCIAL_TIPS document only

Component 4 – Savings Calculator
Input: Monthly income (), $Monthlyexpenses()$, Savings goal ($)
Output: Months to goal, weekly amount, daily amount, savings rate %

Component 5 – Gradio UI
Input: All of the above through four browser tabs
Output: Formatted results displayed in the browser app

Context: I needed to document what each of the five components takes as input and produces as output.

My Prompt: "give me a complete inputs and outputs table for all five components of my smart finance assistant: csv analysis, chatbot, rag, savings calculator, gradio ui"

AI Response Summary: AI gave a table with vague descriptions like "CSV file" as input and "some numbers" as output.

My Critique/Improvement: I asked for specific column names, the exact format of the Amount field including dollar signs and negatives, and concrete example rows from my March-May 2026 data.

Result: Complete table with specific details for all five components including an example input row (2026-03-05, $69.99, Gaming, Xbox Game Pass) and example output format with percentages.

Reflection: Vague prompts get vague answers. Asking for specific column names and examples produced something I could put directly in the notebook Step 2 cell.

Entry 5 – Manual Category Grouping

Artifact:

show me how to group these transactions by category by hand: Gaming $69.99, Gym $60.00, Cinema $17.00, Tea $8.50, Refund -$15.00

Total all amounts:
$69.99 + $60.00 + $17.00 + $8.50 + (-$15.00) = $140.49

Percentages:
Gaming = 69.99 / 140.49 * 100 = 49.8%
Gym = 60.00 / 140.49 * 100 = 42.7%
Cinema = 17.00 / 140.49 * 100 = 12.1%
Tea & Café = 8.50 / 140.49 * 100 = 6.1%
Refund = 15.00 / 140.49 * 100 = 10.7%

the refund must not be included in the positive total or the percentage calculations, refunds are negative and should be separated, can you redo it with refund excluded from the denominator

You're right — refunds should be separated. Here is the correct calculation:

Step 1 – Separate positive spending from refunds:
Positive: $69.99 + $60.00 + $17.00 + $8.50 = $155.49
Refunds: $15.00
Net: $155.49 - $15.00 = $140.49

Step 2 – Percentages using positive total $155.49 only:
Gaming = 69.99 / 155.49 * 100 = 45.0%
Gym = 60.00 / 155.49 * 100 = 38.6%
Cinema = 17.00 / 155.49 * 100 = 10.9%
Tea & Café = 8.50 / 155.49 * 100 = 5.5%

Context: Before coding analyse_spending_patterns I traced the groupby logic manually to understand refund separation.

My Prompt: "show me how to group these transactions by category by hand: Gaming $69.99, Gym$60.00, Cinema $17.00, Tea$8.50, Refund -$15.00"

AI Response Summary: AI included the refund row in the total denominator giving wrong percentages that exceeded 100%.

My Critique/Improvement: I pointed out the refund must be excluded from positive spending before calculating percentages.

Result: Correct calculation showing positive total $155.49, Gaming$45.015.00, net $140.49.

Reflection: The AI made the exact bug I was trying to prevent. Catching it in Step 3 meant the test assert abs(pct_sum - 100.0) < 0.5 confirmed the fix in Step 6.

Entry 6 – Pseudocode for analyse_spending_patterns

Artifact:

write pseudocode for analyse_spending_patterns, separate refunds before grouping, use positive total for percentages, return a dict with summary total refunds net avg n and top

FUNCTION analyse_spending(df):
SPLIT df into positive and refunds
total = SUM of all Amount_Clean
GROUP by Category
ADD percentage column
SORT descending
RETURN dict

the function must be called analyse_spending_patterns not analyse_spending, and you forgot the top key in the return dict — top is the name of the highest spending category and is used by the recommendations function

FUNCTION analyse_spending_patterns(df):

SET positive to rows where Amount_Clean > 0
SET refunds to rows where Amount_Clean < 0

SET total_spent   = SUM of positive Amount_Clean
SET total_refunds = ABSOLUTE VALUE of SUM of refund Amount_Clean
SET net_spent   = total_spent - total_refunds

GROUP positive by Category:
Total_Spent   = SUM
Avg_Transaction = MEAN
Count   = COUNT
ROUND to 2 decimal places

ROUND to 2 decimal places
RESET index so Category is a regular column

ADD Percentage column = Total_Spent / total_spent * 100
SORT by Total_Spent DESCENDING

RETURN {
summary: the sorted DataFrame,
total: total_spent rounded to 2dp,
refunds: total_refunds rounded to 2dp,
net: net_spent rounded to 2dp,
avg: mean of positive amounts,
n: count of positive rows,
top: name of first Category in sorted summary
}

Context:I needed pseudocode with the correct function name before converting to Python..

My Prompt: "write pseudocode for analyse_spending_patterns, separate refunds before grouping, use positive total for percentages, return a dict with summary total refunds net avg n and top"

AI Response Summary: AI wrote the function as analyse_spending and forgot the top key in the return dict.

My Critique/Improvement: I corrected the name to analyse_spending_patterns and asked for the top key which is used by generate_financial_recommendations and by full_report.

Result: Complete pseudocode with correct function name, refund separation before grouping, percentage using positive total only, and return dict including summary, total, refunds, net, avg, n, top.

Reflection: Getting the function name right in pseudocode means it is correct from the start in Step 5. The top key was something AI skipped that was essential for the report and recommendations.

Entry 7 – Pseudocode for chat_finbot Without Fallback

Artifact:

write pseudocode for chat_finbot using get_response from hands-on-ai, optional spending context appended to system prompt, must not crash when server is unavailable

FUNCTION chat_finbot(user_msg, spending_ctx):
SET system = SYSTEM_PROMPT
IF spending_ctx: APPEND to system
TRY:
reply = get_response(user_msg, system=system)
CATCH Exception:
reply = "Error occurred"
RETURN reply

the error message "Error occurred" is useless to the user, can you show a detailed message that lists the possible causes so they know what to check

FUNCTION chat_finbot(user_msg, spending_ctx empty string):

SET system to SYSTEM_PROMPT
IF spending_ctx is not empty:
APPEND "\n\nUser spending summary:\n" + spending_ctx to system

TRY:
SET reply to get_response(user_msg, system=system)
IF reply is empty:
SET reply to "Sorry, I couldn't generate a response."

```
CATCH any Exception as e:
SET reply to multiline message:
"❌ FinBot connection issue."
""
"Possible causes:"
"- Internet issue"
"- API key invalid"
"- HandsOnAI server unavailable"
""
"Technical error: {e}"

ADD (user_msg, reply) to chat_history
RETURN reply
```

Context: I needed pseudocode for chat_finbot that handles errors gracefully without needing a separate Anthropic fallback.

My Prompt: "write pseudocode for chat_finbot using get_response from hands-on-ai, optional spending context, must not crash when server is unavailable"

AI Response Summary: AI wrote a try/except but showed a vague error message like "error occurred".

My Critique/Improvement: I asked for a detailed error message that lists possible causes — internet issue, invalid API key, server unavailable — so the user knows what to check.

Result: chat_finbot with detailed error message listing all possible causes and the technical error detail. No fallback needed because OpenRouter is reliable.

Reflection: The detailed error message is better user experience than a vague crash. A user who sees "API key invalid" knows what to fix. A user who sees "error" does not know where to start.

Entry 8 – Pseudocode for rag_lookup Using get_response

Artifact:

write pseudocode for rag_lookup, it passes FINANCIAL_TIPS in the system prompt and uses get_response, not hands_on_ai.rag

FUNCTION rag_lookup(question):
BUILD prompt with FINANCIAL_TIPS
CALL get_response
RETURN answer

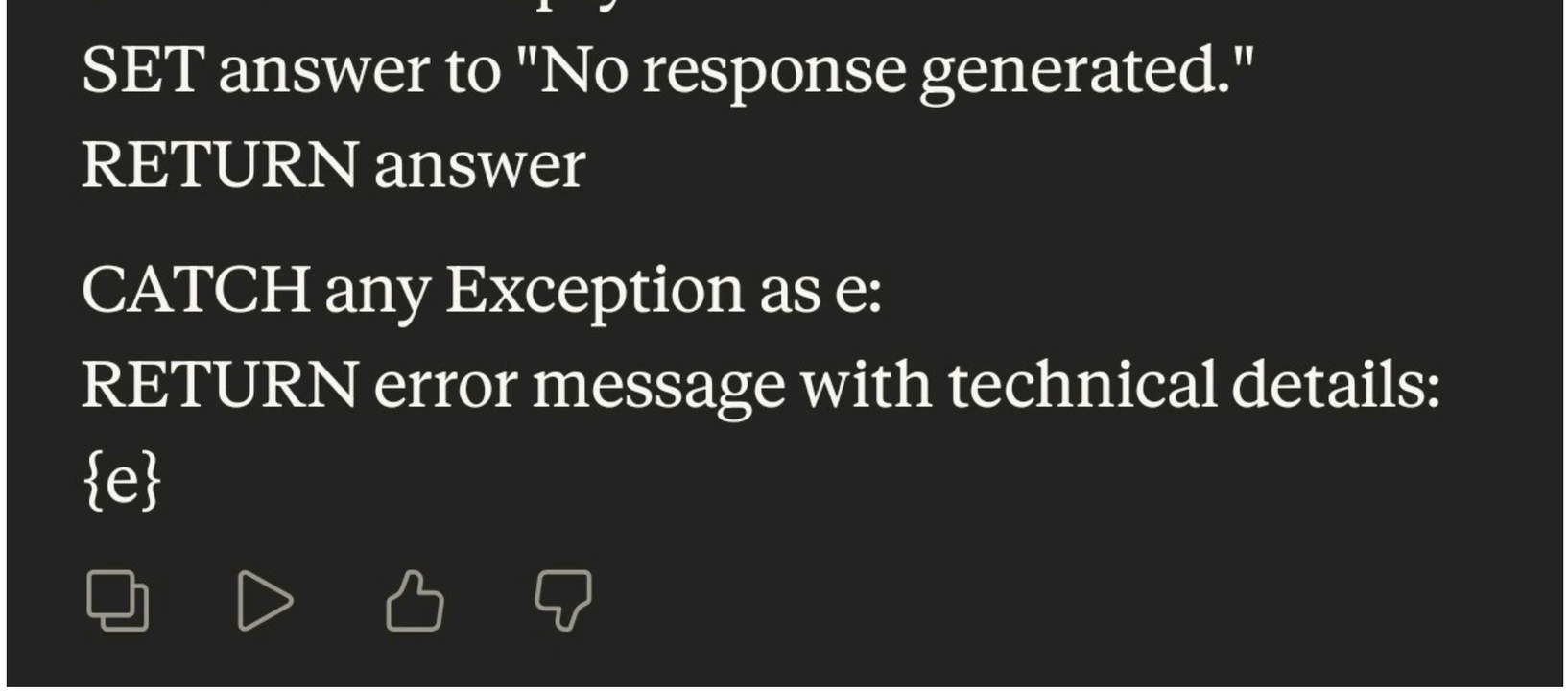
can you confirm that passing the document in the system prompt and telling the model to only answer from it is a valid form of RAG, and show the full pseudocode with proper error handling

Yes — what you're doing is called prompt-based retrieval, a simplified form of RAG. Instead of a vector database, you include the entire document in the system prompt. For a small knowledge base like FINANCIAL_TIPS, this works perfectly well.

FUNCTION rag_lookup(question):

BUILD system_prompt as f-string containing:
"You are a finance assistant."
"Answer ONLY using the document below."
"If the answer is not in the document, say:"
"'I could not find that information in the finance knowledge base.'"
"DOCUMENT:"
{FINANCIAL_TIPS}

TRY:
SET answer to get_response(question, system=system_prompt)
IF answer is empty:



Context: I needed pseudocode for rag_lookup that uses get_response directly instead of hands_on_ai.rag.

My Prompt: "write pseudocode for rag_lookup, it passes FINANCIAL_TIPS in the system prompt and uses get_response, not hands_on_ai.rag"

AI Response Summary: AI gave correct pseudocode and explained why passing the document in the system prompt is a valid form of RAG for small knowledge bases.

My Critique/Improvement: Good answer. I asked AI to confirm the instruction "Answer ONLY using the document below" prevents the model from using general knowledge.

Result: Complete rag_lookup pseudocode with system prompt containing the document and clear instruction to only answer from it, with graceful error handling.

Reflection: Using get_response directly is simpler and more reliable than setting up hands_on_ai.rag for a small knowledge base. The instruction to only answer from the document is what makes it RAG rather than just a regular chat.

Entry 9 – Converting Pseudocode to load_and_clean

Artifact:

convert load_and_clean pseudocode to python, handle gradio file object, raw csv string, and file path, validate four columns, clean dollar signs with regex, drop NaN rows, fillna for Category and Description

Python

```python
def load_and_clean(file_path):
    df = pd.read_csv(file_path)
    df['Amount_Clean'] = pd.to_numeric(
        df['Amount'].str.replace('$', ''), e
    )
    df = df.dropna(subset=['Amount_Clean'])
    return df
```

this only handles file paths, gradio passes a file object with a .name attribute, it also crashes on -$15.00 because of the minus sign before the dollar sign, and it does not validate that the required columns exist

Python

```python
def load_and_clean(file_or_string):
    try:
        if hasattr(file_or_string, 'name'):
            df = pd.read_csv(file_or_string.
        elif isinstance(file_or_string, str)
            df = pd.read_csv(io.StringIO(fil
        else:
            df = pd.read_csv(file_or_string)
    except Exception as e:
        raise ValueError(f"Could not read fi

    required = ['Date', 'Amount', 'Category'
    missing  = [c for c in required if c not
    if missing:
        raise ValueError(f"Missing columns:

    # Regex removes $, commas, and spaces —
    df['Amount_Clean'] = pd.to_numeric(
        df['Amount'].astype(str).str.replace
```

```
        errors='coerce'
    )
    dropped = df['Amount_Clean'].isna().sum(
    if dropped > 0:
        print(f"  Dropped {dropped} row(s) w
    df = df.dropna(subset=['Amount_Clean'])
    df['Category']    = df['Category'].filln
    df['Description'] = df['Description'].fi
    print(f" Loaded {len(df)} transactions s
    return df
```

Context: Converting the load_and_clean pseudocode into Python that handles three input types.

My Prompt: "convert load_and_clean pseudocode to python, handle gradio file object, raw csv string, file path, validate four columns, regex cleaning, drop NaN, fillna"

AI Response Summary: AI only handled file paths and used str.replace(',' ,'' $)whichcrasheson-$15.00.

My Critique/Improvement: I pointed out the three-way input check for Gradio was missing, the regex must handle negative amounts, and errors='coerce' prevents crashes on bad values.

Result: Complete load_and_clean function with hasattr check for Gradio, isinstance for strings, regex r'[$,\s]', errors='coerce', dropna, and fillna.

Reflection: The three-way input check was missing from the AI's first response. Gradio passes file objects with a .name attribute, not plain strings — without the hasattr check the function would crash on every real upload.

Entry 10 – Converting Pseudocode to analyse_spending_patterns

Artifact:

convert analyse_spending_patterns pseudocode to python, named aggregations, percentage column using positive total only, sort descending, return dict with all 7 keys

Python

```
def analyse_spending(df):
    positive = df[df['Amount_Clean'] > 0].co
    refunds  = df[df['Amount_Clean'] < 0].co
    total_spent   = positive['Amount_Clean']
    total_refunds = abs(refunds['Amount_Clea
    summary = (
        positive.groupby('Category')['Amount
        .agg(Total_Spent='sum', Avg_Transact
        .round(2)
    )
    summary['Percentage'] = (summary['Total_
    summary = summary.sort_values('Total_Spe
    return {
        'summary': summary, 'total': round(t
        'refunds': round(total_refunds, 2),
        'net': round(total_spent - total_ref
        'avg': round(positive['Amount_Clean'
        'n': len(positive), 'top': summary.i
    }
```

two problems: the function must be called analyse_spending_patterns not analyse_spending, and you forgot reset_index() after groupby — without it Category is the index not a column and the for loop in full_report will crash with KeyError

Python

```
def analyse_spending_patterns(df):
    positive = df[df['Amount_Clean'] > 0].co
    refunds  = df[df['Amount_Clean'] < 0].co
    total_spent   = positive['Amount_Clean']
    total_refunds = abs(refunds['Amount_Clea
    summary = (
        positive
```

```
        .groupby('Category')['Amount_Clean']
        .agg(Total_Spent='sum', Avg_Transact
        .round(2)
        .reset_index()   # ← essential: make
    )
    summary['Percentage'] = (summary['Total_
    summary = summary.sort_values('Total_Spe
    return {
        'summary': summary,
        'total':   round(total_spent, 2),
        'refunds': round(total_refunds, 2),
        'net':     round(total_spent - total_
        'avg':     round(positive['Amount_Cl
        'n':       len(positive),
        'top':     summary.iloc[0]['Category
    }
```

Context: Converting the analysis pseudocode into Python with correct function name and reset_index.

My Prompt: "convert analyse_spending_patterns pseudocode to python, named aggregations, percentage column using positive total only, sort descending, return dict with all 7 keys"

AI Response Summary: AI called the function analyse_spending and forgot reset_index() after the groupby.

My Critique/Improvement: Corrected the function name to analyse_spending_patterns and explained that without reset_index() the Category column is the index not a regular column, causing KeyError in the for loop.

Result: Correct analyse_spending_patterns function with reset_index() after groupby and all seven keys in return dict.

Reflection: The missing reset_index is a common pandas mistake. After groupby the grouping column becomes the index — reset_index() converts it back to a regular column. Without it the report would crash trying to access row['Category'].

Entry 11 – Converting Pseudocode to chat_finbot

Artifact: